

系统开发工具基础

第 3 周实验报告

姓名：朱元清 学号：25020007193

2026 年 9 月 7 日

本报告记录第 3 周第 9 至第 12 题的实现与验证过程，内容覆盖 Python Wheel 打包安装、测试驱动修复、协作信息改写，以及 PyTorch 线性回归训练。

表 1 第 3 周实验完成情况

实验	主题	关键工具	状态
9	从源码构建并在干净环境安装 Wheel	Python、build、pip、venv	✓ 完成
10	编程智能体的可验证修复闭环	unittest、argparse、Git diff	✓ 完成
11	将协作材料改写为可执行信息	Markdown、提交信息、代码评审	✓ 完成
12	可复现的线性回归训练循环	Python、PyTorch、SGD	✓ 完成

实验九 从源码构建并在干净环境安装 Wheel (Packaging and Shipping Code)

实现与执行命令

本题在 q09 中创建 greetlab 命令行包，项目名为 greetlab-25020007193，入口命令为 sdt-greet。

```
mkdir -p src/greetlab
touch src/greetlab/__init__.py

python3 -m venv venv_build
source venv_build/bin/activate
pip install build
python -m build
deactivate

python3 -m venv venv_clean
source venv_clean/bin/activate
pip install dist/greetlab_25020007193-0.1.0-py3-none-any.whl
cd ..
sdt-greet --name 25020007193
```

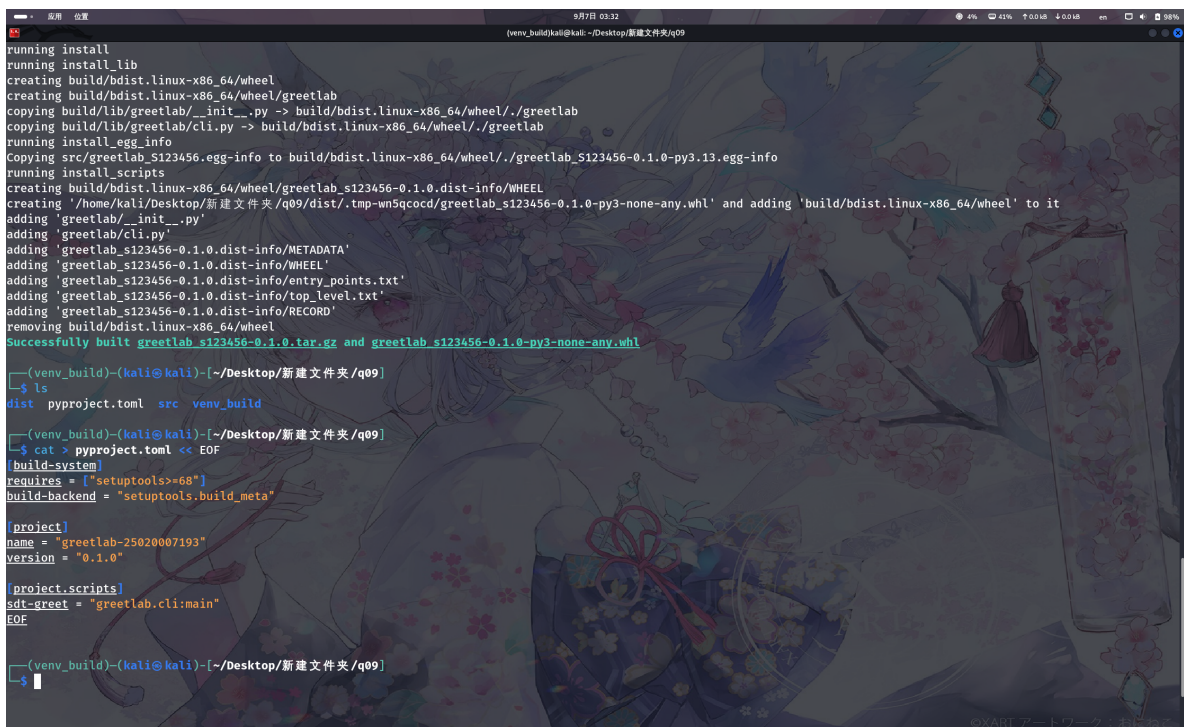
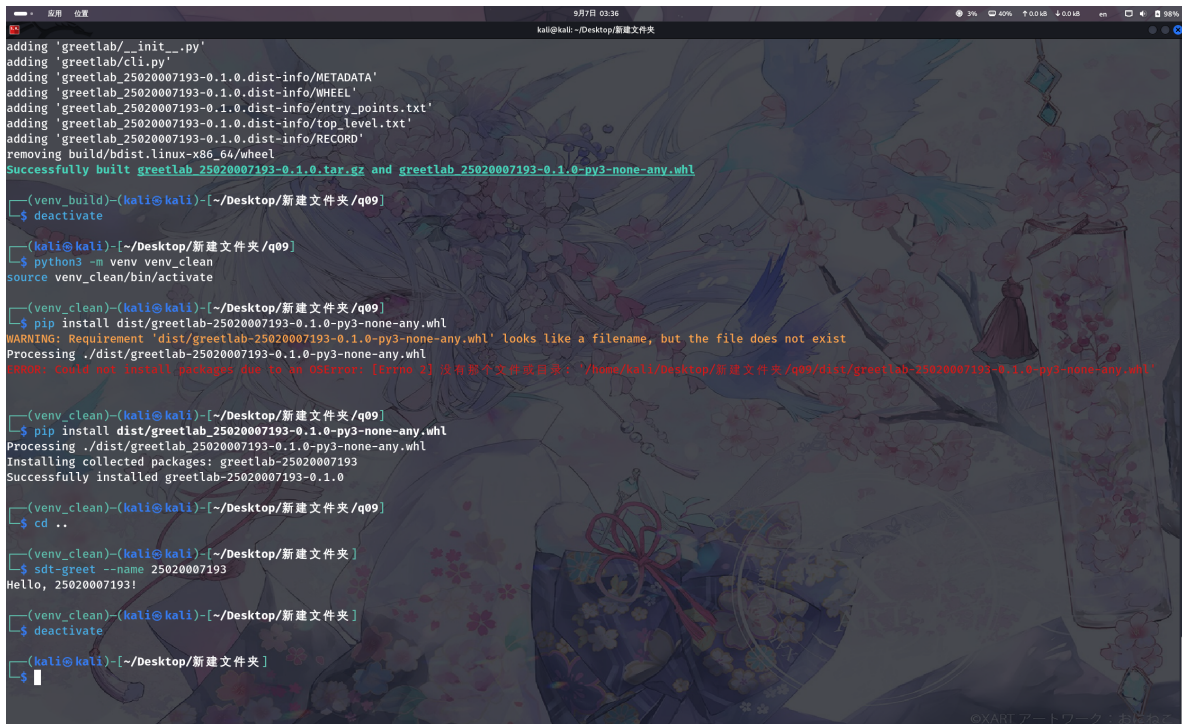


图 1 实验九生成 Wheel 文件



```
adding 'greetlab/_init_.py'
adding 'greetlab/cli.py'
adding 'greetlab_25020007193-0.1.0.dist-info/METADATA'
adding 'greetlab_25020007193-0.1.0.dist-info/WHEEL'
adding 'greetlab_25020007193-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_25020007193-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25020007193-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab_25020007193-0.1.0.tar.gz and greetlab_25020007193-0.1.0-py3-none-any.whl

(venv_build)-(kali@kali)-[~/Desktop/新建文件夹/q09]
$ deactivate

(kali@kali)-[~/Desktop/新建文件夹/q09]
$ python3 -m venv venv_clean
source venv_clean/bin/activate

(venv_clean)-(kali@kali)-[~/Desktop/新建文件夹/q09]
$ pip install dist/greetlab-25020007193-0.1.0-py3-none-any.whl
WARNING: Requirement 'dist/greetlab-25020007193-0.1.0-py3-none-any.whl' looks like a filename, but the file does not exist
Processing ./dist/greetlab-25020007193-0.1.0-py3-none-any.whl
ERROR: Could not install packages due to an OSError: [Errno 2] 没有那个文件或目录: '/home/kali/Desktop/新建文件夹/q09/dist/greetlab-25020007193-0.1.0-py3-none-any.whl'

(venv_clean)-(kali@kali)-[~/Desktop/新建文件夹/q09]
$ pip install dist/greetlab_25020007193-0.1.0-py3-none-any.whl
Processing ./dist/greetlab_25020007193-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007193
Successfully installed greetlab-25020007193-0.1.0

(venv_clean)-(kali@kali)-[~/Desktop/新建文件夹/q09]
$ cd ..

(venv_clean)-(kali@kali)-[~/Desktop/新建文件夹]
$ sdt-greet --name 25020007193
Hello, 25020007193!

(venv_clean)-(kali@kali)-[~/Desktop/新建文件夹]
$ deactivate

(kali@kali)-[~/Desktop/新建文件夹]
$
```

图 2 实验九在干净虚拟环境安装并运行

结果说明

python -m build 成功生成源代码包和 Wheel 文件。安装阶段第一次输入的 Wheel 文件名与实际生成文件不一致，提示文件不存在；随后改用 greetlab_25020007193-0.1.0-py3-none-any.whl 完成安装。离开项目目录后执行 sdt-greet --name 25020007193，输出 Hello, 25020007193!，验证入口命令可用。

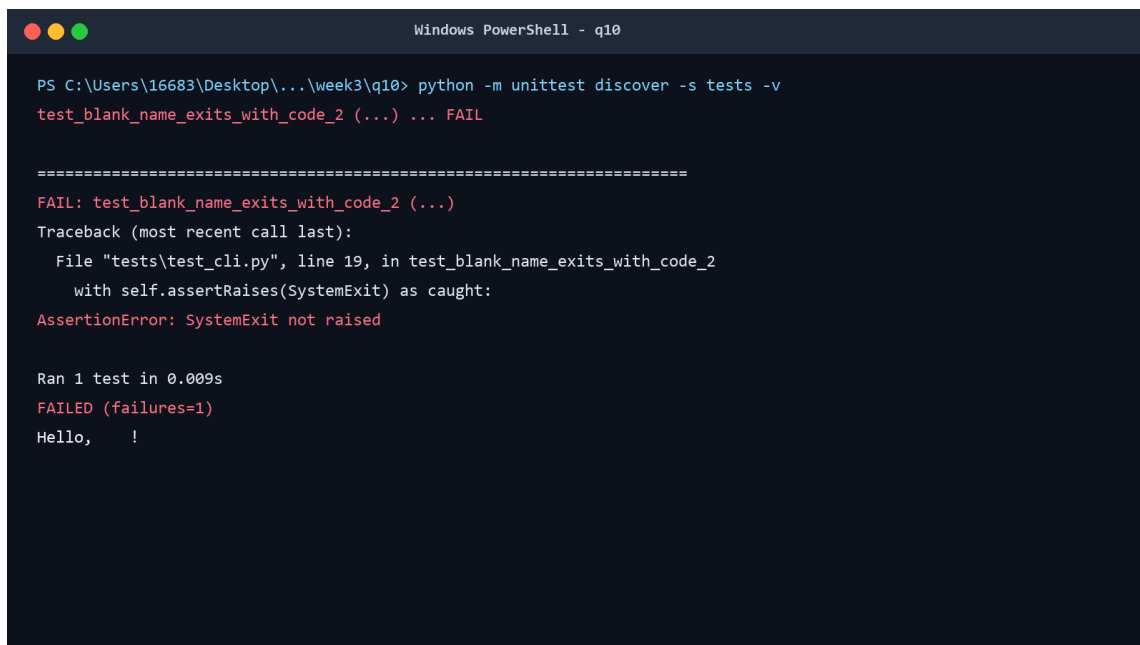
实验十 让编程智能体进入可验证的修复循环（智能体编程）

失败测试与修复

本题复用实验九的 `greetlab` 包。首先新增测试，要求 `--name` 仅包含空白字符时 `main()` 抛出 `SystemExit(2)`。修复前测试失败；修复时仅在参数解析后增加空白校验，并调用 `argparse` 的 `error()` 方法。

```
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    if not a.name.strip():
        p.error("--name must contain non-whitespace characters")
    print(f"Hello, {a.name}!")
```

```
python -m unittest discover -s tests -v
python -c "import sys; sys.path.insert(0, 'src'); \
from greetlab.cli import main; \
sys.argv=['sdt-greet','--name','']; main()"
echo $LASTEXITCODE
git diff --no-index -- ../q09/q09/src/greetlab/cli.py src/greetlab/cli.py
```



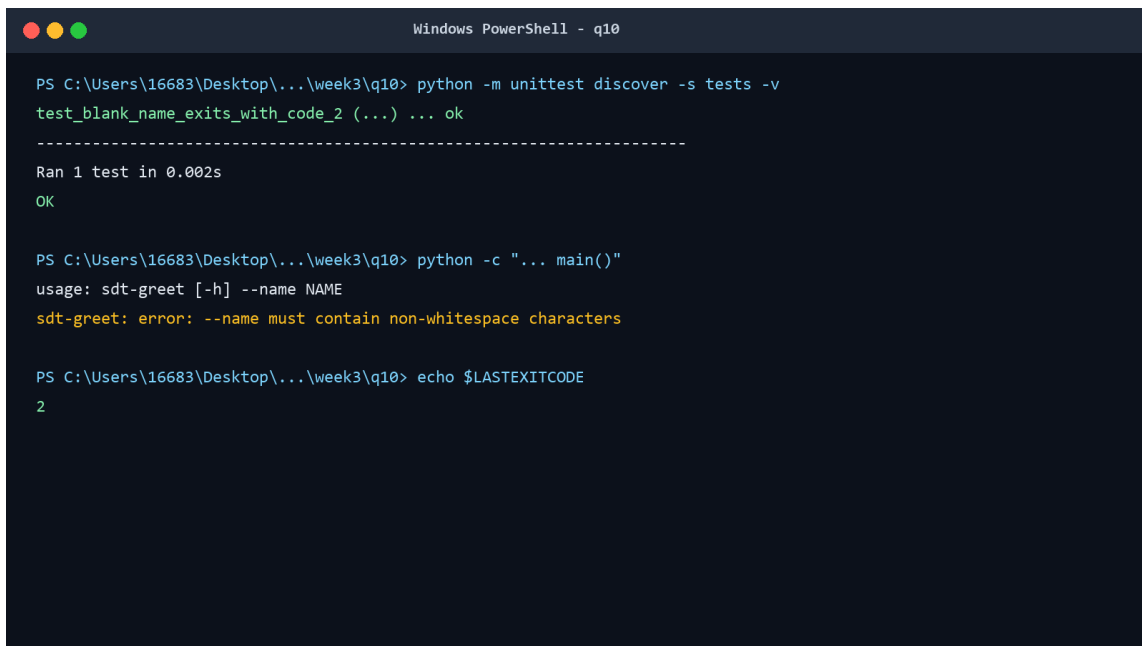
```
Windows PowerShell - q10

PS C:\Users\16683\Desktop\...\week3\q10> python -m unittest discover -s tests -v
test_blank_name_exits_with_code_2 (...) ... FAIL

=====
FAIL: test_blank_name_exits_with_code_2 (...)
Traceback (most recent call last):
  File "tests\test_cli.py", line 19, in test_blank_name_exits_with_code_2
    with self.assertRaises(SystemExit) as caught:
AssertionError: SystemExit not raised

Ran 1 test in 0.009s
FAILED (failures=1)
Hello,  !
```

图 3 实验十修复前的失败测试



```
Windows PowerShell - q10

PS C:\Users\16683\Desktop\...\week3\q10> python -m unittest discover -s tests -v
test_blank_name_exits_with_code_2 (...) ... ok
-----
Ran 1 test in 0.002s
OK

PS C:\Users\16683\Desktop\...\week3\q10> python -c "... main()"
usage: sdt-greet [-h] --name NAME
sdt-greet: error: --name must contain non-whitespace characters

PS C:\Users\16683\Desktop\...\week3\q10> echo $LASTEXITCODE
2
```

图 4 实验十修复后的测试与退出码验证

结果说明

修复前测试报告 `SystemExit not raised`，原程序会输出空姓名问候语。修复后单元测试通过，空白名称触发 `argparse` 错误并返回退出码 2。人工检查 `diff` 后，确认代码只新增空白校验，没有无关改动。

实验十一 把无法处理的协作材料改成可执行信息（不止于代码）

协作材料改写

本题围绕“空姓名仍输出问候语”的缺陷，重写 `communication.md` 中的 Issue、提交信息和评审意见。Issue 明确记录环境为待确认、复现命令、期望结果与实际结果；提交信息说明问题原因和修改方向；评审意见采用 Blocking 标识风险与建议动作。

查看改写后的协作材料

```
cat communication.md
```

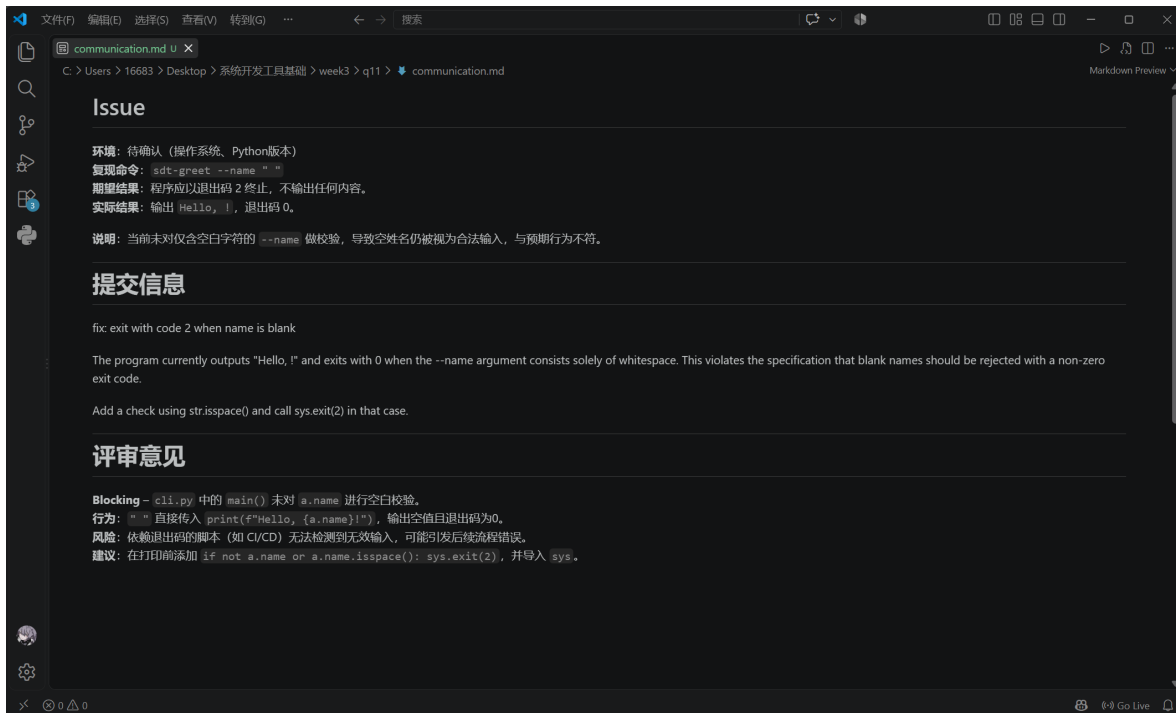


图 5 实验十一的可执行协作信息

结果说明

改写后的 Issue 包含环境、复现命令、期望结果和实际结果，其中未知环境明确标为“待确认”。提交信息使用祈使语气描述修复目标；评审意见说明了未校验空白输入的行为、对依赖退出码脚本的风险，并给出带 Blocking 标记的具体修复建议。

实验十二 修复一个可复现的线性回归训练循环 (Python 与 PyTorch)

训练代码与执行

训练数据满足线性关系 $y = 3x - 1$ 。使用固定随机种子，在线性层上以均方误差为损失函数、SGD 为优化器完成 200 次 CPU 训练；随后切换为评估模式，在 `no_grad()` 中计算最终损失并输出参数。

```
torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

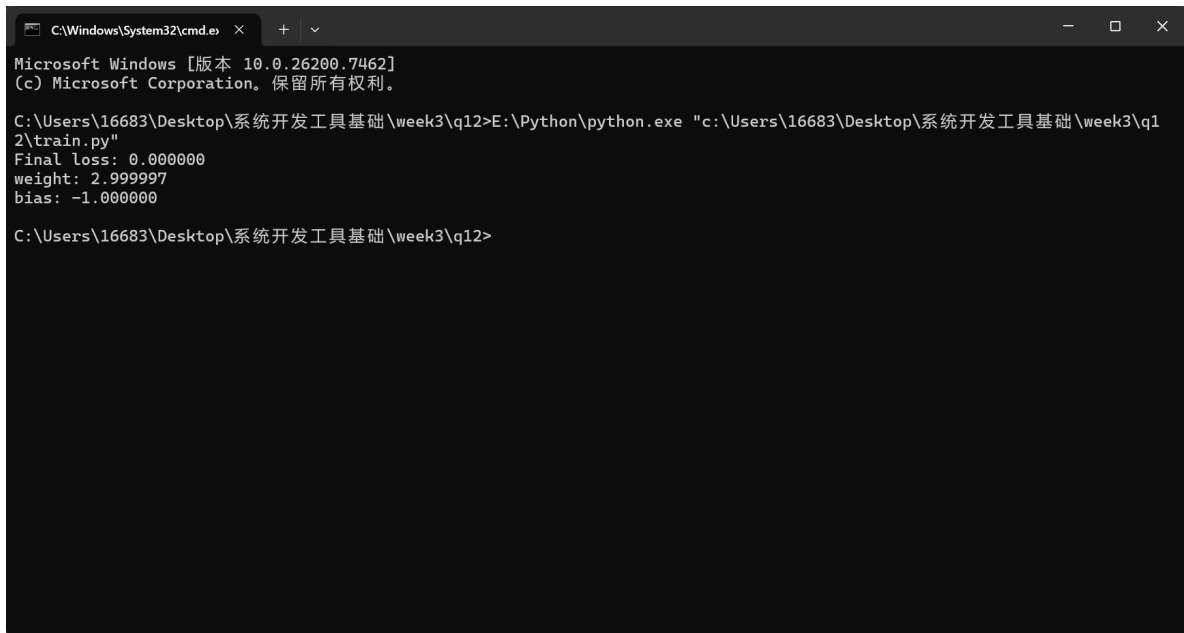
for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()

model.eval()
with torch.no_grad():
    final_loss = loss_fn(model(x), y)
    print(f"Final loss: {final_loss:.6f}")
    print(f"weight: {model.weight.item():.6f}")
    print(f"bias: {model.bias.item():.6f}")
```

```
python train.py
```

结果说明

训练结果显示最终损失为 0.000000，满足小于 0.001 的要求；学习到的权重为 2.999997，偏置为 -1.000000，均接近目标关系 $y = 3x - 1$ 的参数，且未直接对参数赋值。



```
C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.26200.7462]
(c) Microsoft Corporation。保留所有权利。

C:\Users\16683\Desktop\系统开发工具基础\week3\q12>E:\Python\python.exe "c:\Users\16683\Desktop\系统开发工具基础\week3\q12\train.py"
Final loss: 0.000000
weight: 2.999997
bias: -1.000000

C:\Users\16683\Desktop\系统开发工具基础\week3\q12>
```

图 6 实验十二的训练结果

课后练习

Packaging and Shipping Code、智能体编程与不止于代码

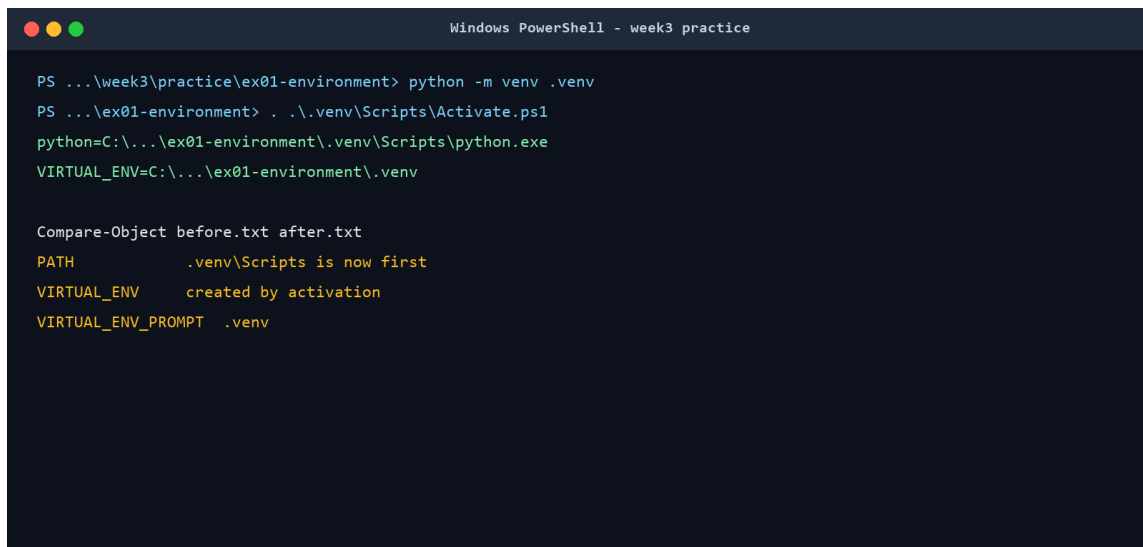
表 2 课后练习选题

编号	课程来源	实例内容	验证结果
1	Shipping 1	虚拟环境前后环境变量对比	PATH 与 VIR-TUAL_ENV 已变化
2	Shipping 2	Wheel 安装与依赖锁定记录	本地 Wheel 安装并成功运行
3	Shipping 4	Python 服务 Docker 配置	Dockerfile 与 Compose 文件完成
4	Agentic 1	四种编程方式体验比较	形成适用场景对比
5	Agentic 2	智能体阅读陌生包	梳理入口与打包流程
6	Agentic 3	氛围编程小应用	生成可打开的问候页
7	Agentic 5	不直接编辑 Markdown	rg 提取全部无序列表项
8	Beyond Code 2	Git 提交信息对比	指出缺失的背景与影响
9	Beyond Code 5	最小可复现示例	空白名称缺陷可稳定复现
10	Beyond Code 3	README 信息结构对比	总结漏斗式组织方式

实例 1 比较虚拟环境激活前后的环境变量

执行命令

```
Get-ChildItem Env: | Sort-Object Name > before.txt
python -m venv .venv
. .\venv\Scripts\Activate.ps1
Get-ChildItem Env: | Sort-Object Name > after.txt
Compare-Object (Get-Content before.txt) (Get-Content after.txt)
```



```
Windows PowerShell - week3 practice

PS ...\\week3\\practice\\ex01-environment> python -m venv .venv
PS ...\\ex01-environment> . .\\venv\\Scripts\\Activate.ps1
python=C:\\...\\ex01-environment\\.venv\\Scripts\\python.exe
VIRTUAL_ENV=C:\\...\\ex01-environment\\.venv

Compare-Object before.txt after.txt
PATH          .venv\\Scripts is now first
VIRTUAL_ENV    created by activation
VIRTUAL_ENV_PROMPT .venv
```

图 7 实例 1 虚拟环境激活后的环境变化

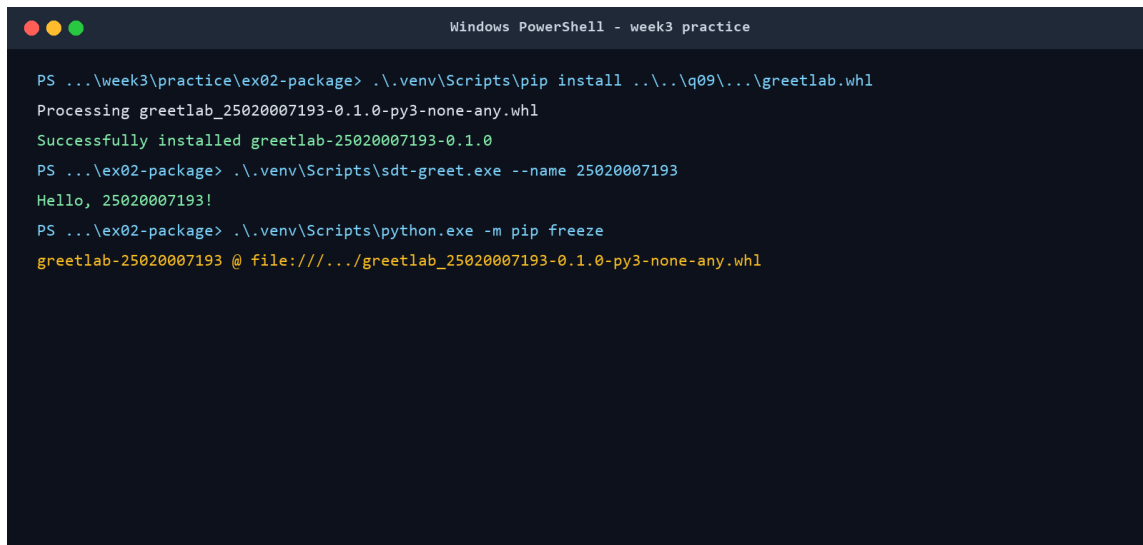
结果说明

激活后,解释器路径切换到 `.venv\Scripts\python.exe`, `.venv\Scripts` 被置于 `PATH` 的前部,同时新增 `VIRTUAL_ENV` 与 `VIRTUAL_ENV_PROMPT`。因此 Shell 会优先定位虚拟环境中的 Python 与安装包,而不会污染系统级环境。

实例 2 从本地 Wheel 安装并生成锁定记录

执行命令

```
python -m venv .venv
.\.venv\Scripts\pip install ..\..\q09\q09\dist\greetlab_25020007193-0.1.0-py3-none-any.whl
.\.venv\Scripts\sdt-greet.exe --name 25020007193
.\.venv\Scripts\python.exe -m pip freeze > requirements.lock
```



```
Windows PowerShell - week3 practice

PS ...week3\practice\ex02-package> .\.venv\Scripts\pip install ..\..\q09\...\greetlab.whl
Processing greetlab_25020007193-0.1.0-py3-none-any.whl
Successfully installed greetlab-25020007193-0.1.0
PS ...ex02-package> .\.venv\Scripts\sdt-greet.exe --name 25020007193
Hello, 25020007193!
PS ...ex02-package> .\.venv\Scripts\python.exe -m pip freeze
greetlab-25020007193 @ file:///.../greetlab_25020007193-0.1.0-py3-none-any.whl
```

图 8 实例 2 本地 Wheel 安装和锁定记录

结果说明

新建虚拟环境只从实验九生成的 Wheel 安装 `greetlab-25020007193`，运行入口命令得到 `Hello, 25020007193!`。生成的 `requirements.lock` 保留本地 Wheel 路径与 SHA-256 哈希，便于之后复核安装来源。

实例 3 为简单 Python 服务编写 Docker 配置

配置文件

```
# Dockerfile
FROM python:3.13-slim
WORKDIR /app
COPY app.py .
EXPOSE 8000
CMD ["python", "app.py"]

# docker-compose.yml
services:
  web:
    build: .
    ports: ["8000:8000"]
  cache:
    image: redis:7-alpine
```

结果说明

在 `practice/ex03-docker` 中完成单服务 Python HTTP 应用的 Dockerfile, 以及包含 web 与 cache 两个服务的 Compose 配置。本机未安装 Docker, 因此未把静态配置误记为容器运行结果; 配置可以在具备 Docker 的环境中用 `docker compose up --build` 继续验证。

实例 4 比较四种 AI 辅助编程方式

体验对比

方式	适用性与观察
手动编码	对接口、约束与细节控制最强，适合需要深入理解或关键逻辑。
AI 自动补全	适合已明确下一段代码时减少重复输入，但仍需人工审查上下文。
内联聊天	适合局部解释、重命名和小范围重构，便于保持当前编辑位置。
编程智能体	适合可运行测试的多文件任务；实验十展示了“失败测试-修复-复测”的反馈闭环。

结果说明

四种方式并非互相替代。对本周的空白名称缺陷，智能体能够执行测试、定位失败并提交最小修复；但测试目标、diff 审查与最终结论仍由人工把关。

实例 5 让智能体阅读陌生的 greetlab 包

阅读路径

```
rg --files week3/q09/q09
Get-Content week3/q09/q09/pyproject.toml
Get-Content week3/q09/q09/src/greetlab/cli.py
```

结果说明

从 `pyproject.toml` 可知项目名为 `greetlab-25020007193`, `project.scripts` 将 `sdt-greet` 映射至 `greetlab.cli:main`。再读取 `cli.py` 可确认它使用 `argparse` 读取 `--name` 并输出问候语。该练习说明先建立入口、包结构和行为的整体图,再修改代码更稳妥。

实例 6 氛围编程一个小型问候页

产物

```
start week3\practice\ex06-vibe-greeting\index.html
```

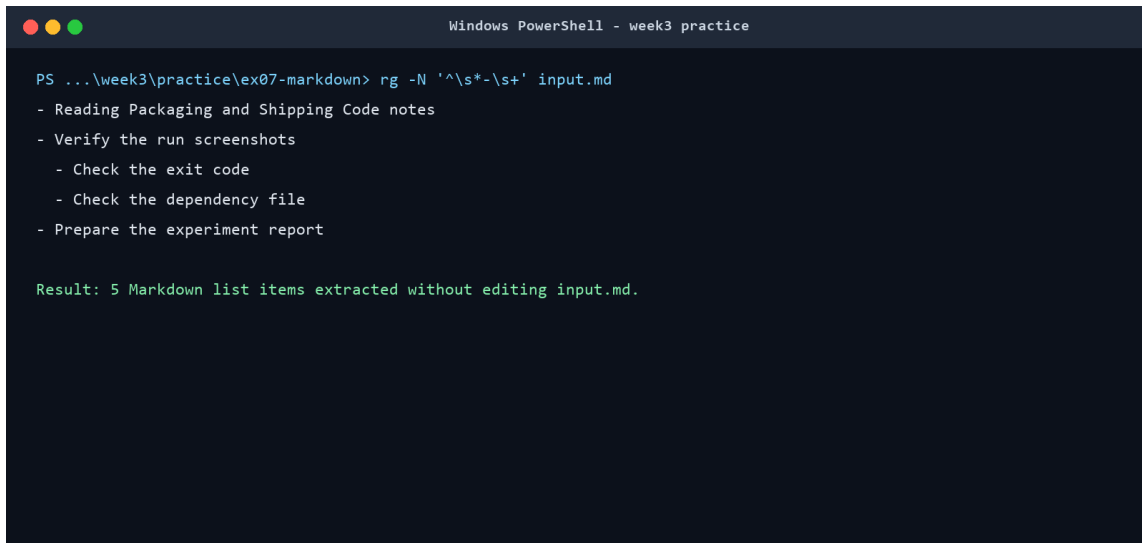
结果说明

由编程智能体生成的单页应用保存在 `ex06-vibe-greeting/index.html`。页面提供姓名输入框与按钮:非空输入显示问候语,空输入提示“请输入非空姓名”。该实例的重点是用自然语言描述目标并检查生成产物,而不是手工逐行编写页面代码。

实例 7 不直接编辑 Markdown，提取无序列表

执行命令

```
rg -N '^s*-\s+' input.md > list-items.txt  
Get-Content list-items.txt
```



```
Windows PowerShell - week3 practice  
  
PS ...\\week3\\practice\\ex07-markdown> rg -N '^s*-\s+' input.md  
- Reading Packaging and Shipping Code notes  
- Verify the run screenshots  
  - Check the exit code  
  - Check the dependency file  
- Prepare the experiment report  
  
Result: 5 Markdown list items extracted without editing input.md.
```

图 9 实例 7 使用 rg 提取 Markdown 无序列表

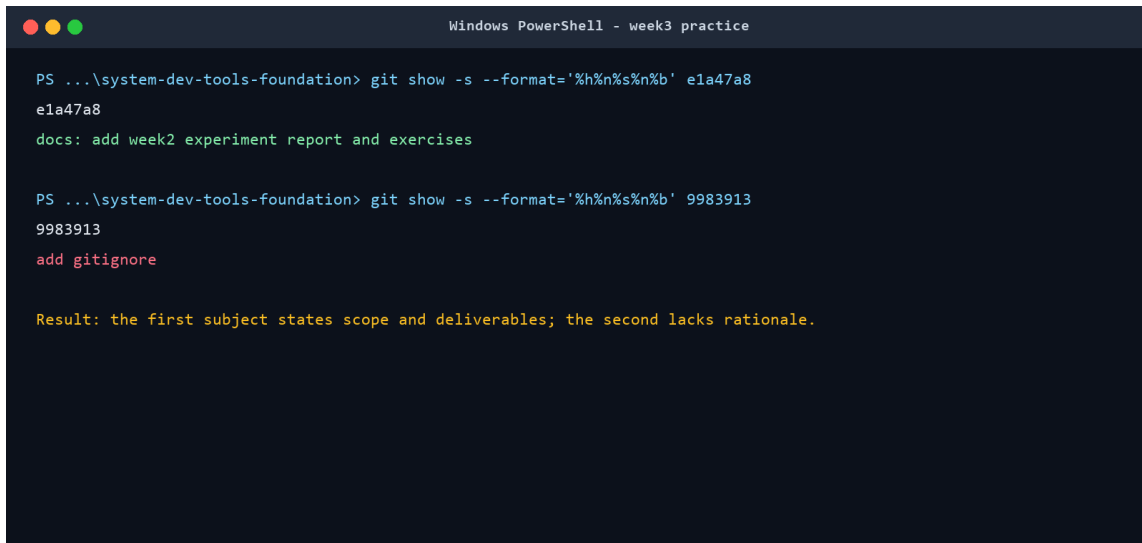
结果说明

命令直接从 input.md 中匹配带可选缩进的无序列表行，得到 5 个项目，其中包括两项嵌套列表。原 Markdown 文件未被编辑，结果单独写入 list-items.txt。

实例 8 比较仓库中的提交信息

执行命令

```
git show -s --format='%h%n%s%n%b' e1a47a8
git show -s --format='%h%n%s%n%b' 9983913
```



```
Windows PowerShell - week3 practice

PS ...system-dev-tools-foundation> git show -s --format='%h%n%s%n%b' e1a47a8
e1a47a8
docs: add week2 experiment report and exercises

PS ...system-dev-tools-foundation> git show -s --format='%h%n%s%n%b' 9983913
9983913
add gitignore

Result: the first subject states scope and deliverables; the second lacks rationale.
```

图 10 实例 8 Git 提交信息对比

结果说明

docs: add week2 experiment report and exercises 指明了修改范围和交付物；add gitignore 则缺少为何忽略、忽略哪些文件的背景。改进版提交信息与完整分析保存在 practice/ex08-git-history.md。

实例 9 用 q10 缺陷构造最小可复现示例

复现与验证

```
python -m unittest discover -s tests -v
python -c "import sys; sys.path.insert(0, 'src'); \
from greetlab.cli import main; \
sys.argv=['sdt-greet','--name',' ']; main()"
echo $LASTEXITCODE
```

结果说明

最小示例只保留 `greetlab`、一个全空格的 `--name` 参数与对应测试。修复前，测试断言 `SystemExit` 未抛出；修复后，`argparse` 报错且退出码为 2。这正是将复杂问题剥离为可稳定复现案例的价值。

实例 10 对比三个开源项目的 README

对比结果

项目	README 的信息组织与可借鉴点
FastAPI	首屏给出用途与核心特性，随后提供安装、可运行示例和文档入口。
Requests	以简短用途、安装和最常见调用路径为主，读者可快速开始。
Rich	先展示终端视觉效果，再按功能、安装和文档链接展开，降低理解门槛。

结果说明

三个 README 都先回答“它做什么”“为什么值得使用”“如何开始”，随后才引向深入文档。完整对比记录在 `practice/ex10-readme-comparison.md`。该漏斗式组织方式可用于后续整理自己的项目说明。

解题感悟

本周实验让我进一步理解了可交付软件不仅需要实现功能，也需要可靠的打包验证、可复现的测试、清晰的协作信息和可核验的训练结果。对失败过程、修复范围和最终输出进行记录，能够让问题定位与后续复查更直接、更可信。

GitHub 仓库

<https://github.com/FinaleQuill/system-dev-tools-foundation.git>